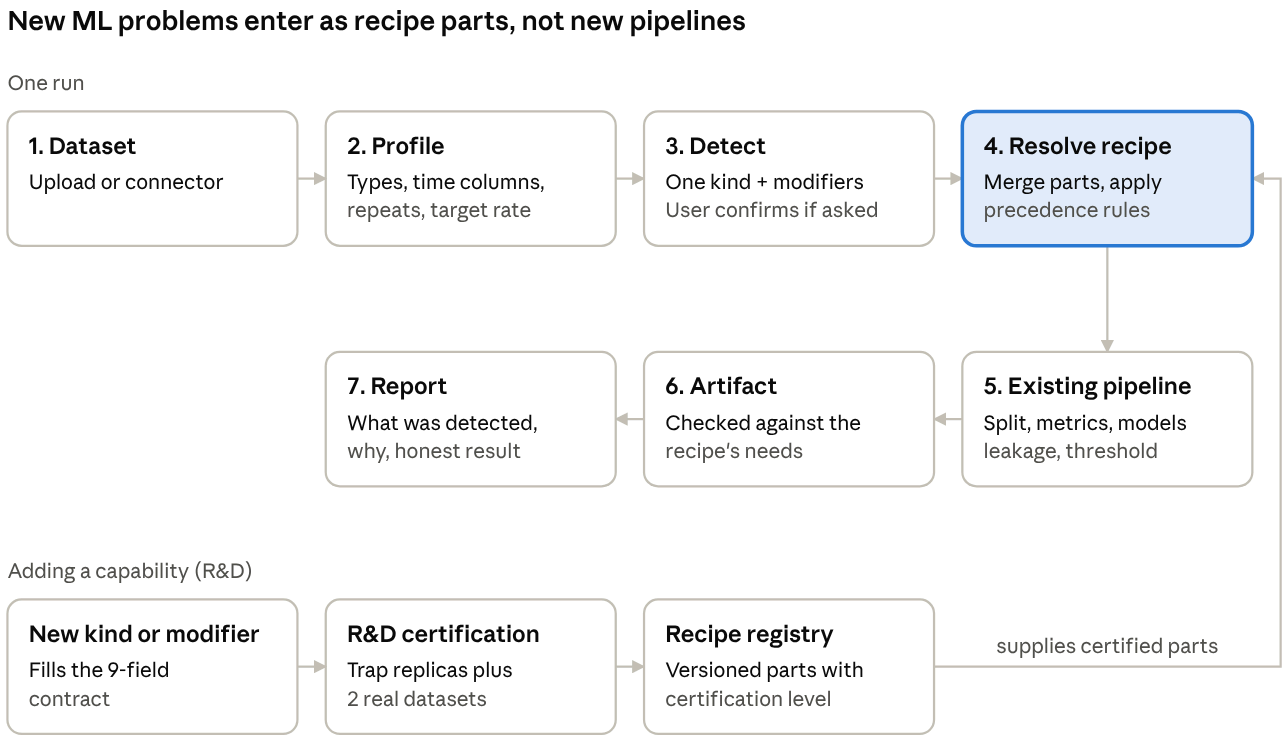
DCLab Task Recipes: Adding Support for New ML Problems

Oct 16, 2026 · @Shahriar

Overview

DCLab should support new kinds of ML problems by adding **recipe parts**, not new pipelines. Every problem is described as one *task kind* (what is predicted) plus zero or more *modifiers* (conditions on the data, such as rare positives or time order). Each part fills the same contract, and each is certified in R&D on trap replicas before the product uses it.



how a run uses a recipe, and how a new capability is added

The top two rows are one run: the recipe only answers questions the existing pipeline already asks. The bottom row is how a new capability gets in: it is written against the contract, certified, and only then placed in the registry the run reads from.

Why this matters: seven kinds combined with a handful of modifiers cover far more real problems than building use cases one by one. Every run can also explain each choice it made, and which part made it.

Core terms

Five terms carry the whole design. Everything else in this doc uses them.

Term	Meaning	Example
Task kind	What the model predicts. Exactly one per problem.	Binary classification, forecasting
Modifier	A condition on the data that changes how a kind must be handled. Zero or more per problem.	Rare positives, time order, duplicate groups
Recipe	The resolved plan for one problem: one kind plus its modifiers, turned into concrete choices for split, metrics, models, checks, threshold and artifact.	Binary + rare + groups + time
Contract	The fixed list of fields every kind and every modifier must fill. It is what makes recipes comparable and testable.	Detection, split, metrics, artifact needs
Certification level	How far a recipe has been proven. Only certified recipes run in the product by default.	Experimental, certified, product default

One rule follows from this: **a new ML problem is never added as a new pipeline.** It is added as a new kind or a new modifier that fills the contract and passes certification.

Task kinds

Seven kinds cover almost all tabular business problems. Each kind sets the defaults below; modifiers then change them.

Kind	Target	Default split	Primary metric	Mandatory baseline	First model families
Binary classification	Two classes	Stratified random	ROC-AUC (PR-AUC if rare)	Logistic regression	LR, RandomForest, LightGBM, XGBoost
Multiclass classification	3+ classes	Stratified random	Macro-F1, log loss	Multinomial LR	LR, LightGBM, XGBoost
Regression	A number	Random	MAE or RMSE (chosen by business cost)	Linear / ridge regression	Ridge, RandomForest, LightGBM
Forecasting	A number per entity per future time step	Rolling-origin backtest	MASE or WAPE	Seasonal naive (same period last cycle)	LightGBM on lags, classic seasonal models
Ranking	An order of items per query or user	Group by query or user	NDCG@K, precision@K	Popularity or score sort	LightGBM ranker
Anomaly scoring	None, or very few labels	Temporal holdout plus audit sample	Precision@K on reviewed cases	Robust z-score per peer group	Isolation Forest, autoencoder, peer comparison
Matching / similarity	Pairs: same or not	Group by entity	Pair precision and recall	Exact key join	Blocking + rules, then embeddings and nearest-neighbour

Kinds 1 to 3 are close to what DCLab does today. Kinds 4 to 7 need new code in the split, metric and artifact steps, not only new model families.

Modifiers

Modifiers are where most real-world difficulty lives. Each one is detected from the data or stated by the user, and each changes a small, known set of pipeline steps.

Modifier	Detected by	What it changes
Rare positives	Minority class under about 5%	Primary metric becomes PR-AUC or precision@K; class weights; stratified folds; a warning whenever accuracy is shown

Modifier	Detected by	What it changes
Groups / duplicates	An ID repeats across rows, or near-identical rows exist	Group split; a check that no group sits in both train and holdout; group-level features allowed
Time order	A date column, and target rate changes over time	Temporal split; features built only from data before each row's cutoff; backtest across several periods
Noisy or weak labels	A label-source column, or a target built from rules	Evaluate only on confirmed labels; label source used as a sample weight; report agreement between sources
Review capacity K	The user enters how many cases can be checked per period	Threshold set at the K-th highest score; precision@K at K becomes primary
Text columns	Long free-text fields	Text becomes its own feature group (TF-IDF first, embeddings later); checked for leakage like any column
Drift expected	Strong change between early and late periods in the profile	Holdout is the latest period; monitoring thresholds written into the artifact
No target	The user says so, or no label column exists	Kind switches to anomaly scoring; a review queue is created

Precedence when several modifiers apply. Splits combine instead of overriding: time order and groups together give a temporal split that also keeps each group on one side. For metrics, the most specific modifier wins: review capacity beats rare positives, which beats the kind's default. A conflict the rules cannot settle stops the run and asks the user.

The recipe contract

Every kind and every modifier fills the same nine fields. A kind fills them all; a modifier fills only the fields it changes and leaves the rest to the kind.

1. **Detection:** the data signals that point to it, with thresholds. Output is a proposal with a confidence, never a silent switch.

2. **Required inputs:** what must exist or be asked for, such as target, time column, group key or capacity K.
3. **Split:** random, group, temporal, or group plus temporal, with fold count and holdout rule.
4. **Metrics:** one primary metric and the guard metrics that must not get worse.
5. **Models:** the mandatory baseline and the allowed model families.
6. **Leakage checks:** checks specific to the kind, on top of the general ones.
7. **Post-processing:** calibration method and threshold rule.
8. **Artifact needs:** the fields `is_self_sufficient` must check for this kind.
9. **Report:** what the user must see to trust the result.

A sketch of the contract as Python dataclasses (a proposal; names are not final):

```
from dataclasses import dataclass, field
from typing import Literal

Split = Literal["random", "stratified", "group", "temporal",
"group_temporal", "rolling_origin"]
Level = Literal["experimental", "certified", "product_default"]

@dataclass(frozen=True)
class Detection:
    signal: str            # e.g. "minority_rate"
    rule: str              # e.g. "< 0.05"
    asks_user: bool        # True if a person must confirm

@dataclass(frozen=True)
class RecipePart:          # one kind or one modifier
    name: str              # "binary", "rare_positives", ...
    part_type: Literal["kind", "modifier"]
    version: str
    level: Level
    detection: list[Detection]
    required_inputs: list[str] = field(default_factory=list)
    split: Split | None = None
    primary_metric: str | None = None
    guard_metrics: list[str] = field(default_factory=list)
    baseline: str | None = None
    model_families: list[str] = field(default_factory=list)
    leakage_checks: list[str] = field(default_factory=list)
    calibration: str | None = None
```

```
    threshold_rule: str | None = None
    artifact_needs: list[str] = field(default_factory=list)
    report_items: list[str] = field(default_factory=list)
    certification_replicas: list[str] = field(default_factory=list)

@dataclass(frozen=True)
class ResolvedRecipe:      # what one run actually uses
    kind: RecipePart
    modifiers: list[RecipePart]
    choices: dict          # final split, metrics, families, threshold...
    reasons: dict          # which part decided each choice, and why
```

The `reasons` field matters as much as the choices. It lets every run explain, for example, "temporal split because modifier `time_order` v1 detected a date column and a 30% drift in target rate."

How a recipe plugs into the pipeline

The recipe sits between profiling and training. The pipeline keeps its existing steps; the recipe only supplies the answers those steps already need.

Resolution order:

1. Profile the dataset (types, missing values, time columns, repeated IDs, target rate).
2. Run every kind's and modifier's detection rules on the profile.
3. Propose one kind and a set of modifiers, each with its evidence and confidence.
4. Ask the user only where `asks_user` is true or confidence is low (for example, capacity K is always asked).
5. Merge the parts into a `ResolvedRecipe`, applying the precedence rules from the Modifiers section.
6. Hand each choice to the decision point that owns it, and log it with its reason.

Which decision point each field drives:

Contract field	Existing decision point
Kind	`task.kind`
Target	`target.column`
Split	`split.strategy`

Contract field	Existing decision point
Models	`training.families_budget`
Leakage checks	`feature.leakage_suspect`
Modifiers (new)	One new point per modifier, such as `modifier.rare_positives` and `modifier.time_order`

The new modifier points use the same registry, decision records and three-answer setup (rule, Jev, DCLab's own model) as the existing points. They also become new labelled targets for the decision models.

Rule versus LLM. Rules decide the split, the metric, the threshold and the artifact checks, so a run gives the same result with the LLM turned off. The LLM may propose a kind or modifier ("this looks like time-ordered data"), explain choices to the user, and draft the report. This keeps the deterministic pipeline and the existing "never AI" rules intact.

Artifact contract per kind

Today `is_self_sufficient` checks four things: feature names, feature order, a fitted preprocessor and the target definition. That is correct for classification, but other kinds need more to make a correct prediction on new data. The check should read its required fields from the recipe instead of a fixed list.

Kind or modifier	Extra fields the artifact must carry
All kinds	The four existing fields, plus the recipe name and version
Binary / multiclass	Class labels in order, calibration map, chosen threshold and the rule that set it
Regression	Target transform (for example log) and its inverse
Forecasting	Horizon, frequency, lag and rolling-window recipe, and the last observed window needed to build features
Ranking	Query or group key name, and the cut-off K used in evaluation

Kind or modifier	Extra fields the artifact must carry
Anomaly scoring	Reference statistics per peer group, score direction (high = bad), threshold, and the review capacity it assumes
Matching	Blocking keys, comparison features, and the match threshold
Modifier: groups	Group key column name
Modifier: time order	Cutoff rule for features, training date range
Modifier: drift	Baseline distributions and alert thresholds for monitoring

Test for every kind: save the artifact, load it in a clean process with only the artifact, predict on held-out rows, and compare with the predictions made at training time. They must match. This is the same check that caught the missing scaler bug, applied to every kind.

Certification

A recipe part earns its level in R&D before the product relies on it. This follows the R&D rule that product claims are earned before they ship.

Level	Where it runs	How it gets there
Experimental	R&D only	Contract filled, unit tests pass
Certified	Product, behind an opt-in flag	Passes every pass criterion below on its trap replicas and on at least 2 real datasets
Product default	Product, chosen automatically when detected	Certified, plus no wrong detection on the full R&D dataset set and no regression in twin runs

Trap replicas. Each part ships with at least one synthetic dataset where the right answer is known by construction, built in `dclab_core/replicas`. Each trap tests one thing that commonly goes wrong:

Part	Planted trap	Right behaviour
Rare positives	1% positive rate	Reports PR-AUC or precision@K and warns on accuracy
Groups	Copies of the same entity spread across rows	Group split; holdout score stays close to the honest score
Time order	A feature computed with future data	Flags the feature; temporal split
Noisy labels	Target built from a rule that is also a feature	Evaluates on confirmed labels only; flags the rule feature
Forecasting	Strong seasonality	Beats the seasonal naive baseline, or says it does not
Anomaly scoring	A small set of planted anomalies	Finds most of them in the top K

Pass criteria (all must hold):

1. Detection picks the right kind and modifiers on every trap replica.
2. Every planted trap is caught.
3. The resolved recipe beats or matches the mandatory baseline on holdout, or reports honestly that it does not.
4. The artifact passes the save, reload and predict test.
5. The result is the same with the LLM turned off.

Each certified part also produces twin runs (workflow vs Claude agent) and SFT samples, so R&D coverage grows at the same pace as product coverage.

How to add a new kind or modifier

Follow these steps in order for every new capability. Each step has a clear output, so a coding agent can do one step per task.

- [ ] **1. Decide: kind or modifier.** If the thing being predicted changes, it is a kind. If only the data conditions change, it is a modifier. Most new requests are modifiers.
- [ ] **2. Write the contract.** Fill all nine fields (a modifier fills only the fields it changes). Write the detection rule with numeric thresholds.

- [ ] **3. Build the trap replica.** Add a synthetic dataset to `dclab_core/replicas` with the right answer known by construction, plus its expected-answer file.
- [ ] **4. Add the decision point.** Register `modifier.<name>` (or extend `task.kind`) in the decision registry, so every run records the answer and its source.
- [ ] **5. Implement what changes.** Only the steps the contract names: split, metric, threshold, leakage check, artifact fields. No new pipeline.
- [ ] **6. Extend the artifact check.** Add the part's `artifact_needs` to `is_self_sufficient`, and add the save, reload and predict test.
- [ ] **7. Add the report items.** Show the user what was detected, why, and the honest result against the baseline.
- [ ] **8. Run certification.** Trap replicas plus at least 2 real datasets, against the pass criteria. Record the result with the part's version.
- [ ] **9. Run twin runs and generate SFT samples.** Compare the workflow with the Claude agent on the same replicas; add checked samples to `sft_factory`.
- [ ] **10. Promote.** Experimental → certified → product default, one level at a time, each with its evidence saved.

Definition of done: the part is versioned, certified, explained in the report, carried in the artifact, and the run gives the same result with the LLM turned off.

Worked example: retail returns fraud

This is the case from the Prateek Singh interview. A user uploads POS returns data with a `is_fraud` column and asks DCLab to flag fraud.

What detection finds:

Part	Evidence in the data	Decision
Kind: binary	`is_fraud` has two values	`task.kind` = binary
Rare positives	1.2% of rows are fraud	On
Groups	`bill_id` repeats across sale and return rows; `cashier_id` repeats	On, group key = `bill_id` (user confirms)
Time order	`txn_time` column; fraud rate rises in the last 3 months	On

Part	Evidence in the data	Decision
Noisy labels	`label_source` column: rule, expert, cluster	On, evaluate on `expert` only
Review capacity	Asked: the team checks 300 cases a day	K = 300

The resolved recipe:

- **Split:** temporal and grouped. The last 3 months are the holdout, and no `bill_id` crosses the boundary.
- **Primary metric:** precision@300 per day, with recall on the audit sample as a guard. Accuracy is shown only with a warning.
- **Models:** logistic regression baseline, then LightGBM and XGBoost with class weights. Rule-derived features are checked for leakage against rule-derived labels.
- **Threshold:** the score of the 300th case on an average day, recorded in the artifact.
- **Artifact:** the standard four fields plus the threshold rule, `bill_id` as group key, the feature cutoff rule, and baseline distributions for drift monitoring.
- **Report:** what was detected and why, precision@300 against the baseline, and a note that recall is estimated only from the audit sample.

If the user uploads the same data without `is_fraud`, detection switches the kind to anomaly scoring and creates a review queue. Reviewed cases then become the labels for the binary recipe above. That is the same path Prateek's team took from 0 to 70,000 labels.

Build order

The order puts the cheapest, most useful parts first and keeps the current product priority on top.

0. **Close the upload-to-result loop for binary.** Every recipe plugs into it, so it stays first.
1. **Write the recipe registry and the `binary` kind** under the contract, with no behaviour change. This proves the structure on what already works.
2. **Add three modifiers on binary:** rare positives, groups and time order. They change only split, metric and threshold, and one retail-returns fraud replica certifies all three.
3. **Add noisy labels and review capacity,** to finish the fraud case.
4. **Add multiclass and regression.** The expansion pack already has data for these.

5. **Add forecasting.** The Rossmann adapter already exists.
6. **Add anomaly scoring,** with a review queue that reuses the blind label queue from the Decision Model Plan.
7. **Add matching and ranking** when a pilot needs them. Add text columns as a modifier when a dataset needs it.

Open questions

- [ ] Where should the registry live first: the R&D repo only, or a shared package the product imports from day one?
- [ ] Should detection results above a confidence level apply automatically, or should every modifier be confirmed by the user in the first version?
- [ ] Is K (review capacity) a per-run input, or a workspace setting reused across runs?
- [ ] Which 2 real datasets certify each part? Ideally one public and one from a pilot.

خلاصهٔ فارسی

این سند روش افزودن پشتیبانی از مسئله‌های متنوع به ML در DCLab را توضیح می‌دهد. ایدهٔ اصلی ساده است: هر مسئلهٔ جدید به‌جای یک پایپ‌لاین جدید، به‌شکل یک «نوع وظیفه» یا یک «اصلاح‌گر» اضافه می‌شود. هر دو یک قرارداد ثابت را پر می‌کنند و پیش از ورود به محصول، در R&D گواهی می‌گیرند.

مفهوم	معنا
نوع وظیفه	چیزی که مدل پیش‌بینی می‌کند؛ ۷ نوع: دودویی، چندکلاسه، رگرسیون، پیش‌بینی سری زمانی، رتبه‌بندی، امتیازدهی ناهنجاری، تطبیق رکورد
اصلاح‌گر	شرایط داده که روش کار را تغییر می‌دهد؛ مانند نمونهٔ مثبت نادر، گروه تکراری، ترتیب زمانی، برچسب نویزدار، ظرفیت بررسی
دستورالعمل	ترکیب نهایی یک نوع و اصلاح‌گرهایش برای یک اجرای مشخص
قرارداد	فیلد ثابت: تشخیص، ورودی لازم، تقسیم داده، معیار، مدل، ۹ بررسی نشت داده، آستانه، نیازهای فایل مدل، گزارش
سطح گواهی	آزمایشی، گواهی‌شده، پیش‌فرض محصول

نکته‌های کلیدی:

- فقط پیشنهاد و توضیح می‌دهد و LLM قانون‌های قطعی، تقسیم داده و معیار و آستانه را تعیین می‌کنند؛ نتیجه بدون LLM نیز یکسان باقی می‌ماند.
- باید فیلدهای لازم را از دستورالعمل بخواند؛ برای نمونه، مدل `is_self_sufficient` بررسی پیش‌بینی سری زمانی به افق پیش‌بینی و آخرین پنجرهٔ داده نیاز دارد.
- هر بخش جدید یک دادهٔ تله‌دار در `dclab_core/replicas` دارد و باید همهٔ تله‌ها را پیدا کند تا گواهی بگیرد.
- برای افزودن هر قابلیت جدید، چک‌لیست ۱۰ مرحله‌ای همین سند دنبال می‌شود.

ترتیب ساخت: ۰. تکمیل چرخهٔ «بارگذاری ← نتیجه»؛ ۱. ساخت رجیستری و نوع دودویی؛ ۲. سه اصلاح‌گر نمونهٔ نادر، گروه و ترتیب زمانی؛ ۳. برچسب نویزدار و ظرفیت بررسی؛ ۴. چندکلاسه و رگرسیون؛ ۵. پیش‌بینی سری زمانی؛ ۶. امتیازدهی ناهنجاری؛ ۷. تطبیق و رتبه‌بندی.